LinkNotes: A Personal Knowledge Base with Full-Text Search and Link Graph Ranking

CS349 (DBIS Lab) – Project Report

Mehul Borad	23B0930
Sahil	23B0943
Thanuj	23B1082
Raghav Goyal	23B0908

Repository: https://github.com/Mehul-Borad/Link-Notes-DBIS

1 Motivation

Most note-taking apps use plain keyword matching for search, ignoring *which* notes are central in your personal knowledge base. We wanted to build a wiki-style notes app where notes can link to one another via `[[Note Title]]` syntax, and where search results are ranked by a combination of (i) text relevance and (ii) graph centrality, so that frequently-referenced concepts surface first — the same intuition behind PageRank, applied to a private corpus.

The project also gave us an excuse to push PostgreSQL beyond simple CRUD: we use full-text search via `tsvector` + GIN, recursive CTEs for path-finding, a PL/pgSQL function for the iterative PageRank computation, and a materialized view to cache its result. The deliberate goal was to keep the "interesting computation" inside the database rather than in application code.

2 Functionality Implemented

2.1 Notes & links (baseline)

- Full CRUD on notes: title (unique), content, timestamps. Endpoints `GET/POST/PUT/DELETE /api/notes` **[P1–P6]**.
- `[[Title]]` and `[[Title|Alias]]` wiki-link parsing. On every save, `sync_links()` reparses the content and rewrites the directed-edge rows in the `links` table.
- Backlink listing: `GET /api/notes/{id}` returns the note plus its outgoing links *and* the notes that link to it (joined to titles).
- Sample seed of 10 interconnected CS notes (Operating Systems, B-Trees, etc.) so the graph has interesting structure out of the box **[P7]**.
- React SPA: sidebar of titles, main view, create/edit/delete forms, clickable wiki links (red), broken links shown gray with strike-through **[P8–P11]**.
- Bug fix in the alias regex: `[[Operating Systems|OS]]` previously stored the entire `Operating Systems|OS` string as the lookup key, so the link silently failed; the regex now strips the alias **[P12]**.

2.2 Full-text search

A generated `tsvector` column on `notes` concatenates the title (weight `A`) and the content (weight `B`) using `setweight` + `to_tsvector('english', ...)`, indexed by GIN. `GET /api/search?q=...` returns each match with `ts_rank`, a `ts_headline` snippet, and (see below) a graph centrality score **[P13]**.

2.3 PageRank as a materialized view

A PL/pgSQL function `compute_pagerank(damping, max_iter, tolerance)` runs the standard power-iteration over the `links` graph (defaults: d=0.85, 30 iterations, early-stop when L_1 delta $< 10^{-4}$). State lives in a persistent `pr_state` table; we initially used `TEMP TABLE`s but `REFRESH MATERIALIZED VIEW` runs in a security-restricted context that forbids temp-table creation, so we

switched to an unlogged persistent table and DML-only operations [P14], [P18]. The view `page_rank` stores the raw probability and a normalized $[0, 1]$ score; `POST /api/admin/refresh-rankings` triggers a refresh, and `seed.py` calls it after seeding.

2.4 Combined ranking

The search query joins `page_rank` and computes

$$\text{combined_score} = (1 - w) \cdot \text{ts_rank} \; + \; w \cdot \text{normalized_pagerank}, \quad w = 0.3 \text{ by default}$$

in a single SQL statement, so all the work happens inside one indexed query [P15]. The endpoint also returns the individual scores so the breakdown is visible.

2.5 Orphan detection & shortest path

`GET /api/orphans` returns notes with no incoming or outgoing edges (`NOT EXISTS`). `GET /api/path?from_id=X&t` runs a BFS over the link graph using a *recursive CTE* that carries the visited path as an `INT[]` for cycle detection (`NOT (target = ANY(path))`), capped at depth 8 [P16].

2.6 Frontend features

- **Search bar** in the sidebar; results replace the note list and show the score breakdown. "Orphans" button toggles to an orphan-only view; "Refresh ranks" calls the admin endpoint; per-note "Find path to" form with autocomplete [P17].
- **Search-term highlighting**: matched terms are wrapped in `<mark>` in the open note's title and body, with a wavy underline variant inside `[[wiki-link]]` text so the link styling stays intact [P19].
- **Wiki-link autocomplete**: typing `[[` pops up a dropdown of matching titles (substring match); ↑ / ↓ to navigate, Enter/Tab inserts and auto-closes `]]`, Escape dismisses. Factored into a reusable `NoteEditor` component used by both create and edit modes [P20].
- **Embeds and external links**: markdown-style `![alt](url)` renders an inline image (`.png/.jpg/.jpeg/.gif/` or PDF iframe (`.pdf`); `[text](url)` renders an external link [P21].
- **File uploads**: `POST /api/upload` (multipart) validates extension, caps at 20 MB, stores under `backend/uploads/<uuid>.<ext>`, served via `StaticFiles`; the editor's "Attach" button inserts `![filename](url)` at the cursor on success [P23].
- **Graph view** (`Graph.js`): full link graph rendered with `react-force-graph-2d`. `GET /api/graph` returns nodes (with PageRank score) + directed edges [P22].
- **Rectangle nodes & freeze-on-drag**: nodes are rounded rectangles showing title, "→ N links", "← N backlinks"; arrows terminate at the rectangle edge via a custom `linkCanvasObject`; after the initial layout settles, every node is pinned (`fx/fy`) so dragging one moves only that one — no global jitter [P24].
- **Persistent layout**: every drag-end saves `{id: {x,y}}` to `localStorage`; on mount, saved positions are restored as `fx/fy` before the data is handed to the force engine. A "Reset layout" button clears storage and re-runs the simulation [P25].

3 Database Design

```
-- Core tables
notes(id PK, title UNIQUE, content, created_at, updated_at,
      search_vector tsvector GENERATED ALWAYS AS (
        setweight(to_tsvector('english', title),   'A') ||
        setweight(to_tsvector('english', content), 'B')) STORED)
links(id PK, source_note_id FK -> notes ON DELETE CASCADE,
                target_note_id FK -> notes ON DELETE CASCADE,
```

```
                UNIQUE(source_note_id, target_note_id))

-- Indexes
idx_notes_search    : GIN  on notes(search_vector)
idx_links_source    : btree on links(source_note_id)
idx_links_target    : btree on links(target_note_id)
idx_page_rank_id    : unique btree on page_rank(note_id)
idx_page_rank_score : btree on page_rank(score DESC)

-- Computation
pr_state              : unlogged scratch table for the iteration
compute_pagerank(...) : PL/pgSQL function, returns (note_id, score)
page_rank             : MATERIALIZED VIEW caching normalized scores
```

The `ON DELETE CASCADE` on `links` means deleting a note transparently cleans up its edges. The unique `(source, target)` pair plus an `ON CONFLICT DO NOTHING` insert in `sync_links` keeps that table idempotent.

4 Code Created

The repository is organized as:

```
backend/
  db.py            schema, init_db(), refresh_rankings(), PageRank fn,
     matview
  main.py          FastAPI app: notes CRUD, search, orphans, path,
                   upload, graph, admin/refresh-rankings
  seed.py          10-note CS-topics seed + auto-refresh
  requirements.txt fastapi, uvicorn, psycopg2-binary, pydantic,
                   python-dotenv, python-multipart
  uploads/         user-uploaded images and PDFs (gitignored)
frontend/
  src/App.js       sidebar, note CRUD, search/orphans/path UI,
                   highlighting, NoteEditor with [[ autocomplete and
                   file upload, view toggle
  src/Graph.js     react-force-graph-2d wrapper, rectangle nodes,
                   custom edge-routed arrows, layout persistence
  src/App.css      styling for everything above
PROMPT_LOG.md      25 prompts used during development
report.tex / .pdf  this report
.gitignore         excludes .venv, node_modules, uploads, build, .env
```

Roughly: `db.py` ~130 lines, `main.py` ~300 lines, `App.js` ~580 lines, `Graph.js` ~220 lines.

5 How We Used AI

We used Claude (the `Sonnet` family) for nearly all code generation, then copy-pasted, ran, and modified the output as needed. Every prompt is recorded verbatim in `PROMPT_LOG.md`. Roles in our session:

- **[P1–P4]** *Schema and primitives.* Generating the SQL schema, the psycopg2 connection helper with `init_db`, the wiki-link regex, and `sync_links`.
- **[P5–P7]** *API + seed.* The complete CRUD layer in FastAPI, backlink retrieval, and the 10-note CS-topics seed with cross-references.
- **[P8–P11]** *Frontend baseline.* Sidebar + note CRUD UI, link rendering, and visual styling.
- **[P12]** *Bug fix.* A targeted regex repair for `[[Title|Alias]]`.

- **[P13–P17]** *The DB-heavy core.* Full-text search with weighted tsvector + GIN, the PL/pgSQL PageRank with iteration loop and convergence check, materialized view + refresh endpoint, combined ranking, orphan detection, recursive-CTE BFS for shortest path, and the corresponding React UI.
- **[P18]** *Debugging.* Diagnosing and fixing the "cannot create temporary table within security-restricted operation" error from `REFRESH MATERIALIZED VIEW`, which led us to switch from temp tables to a persistent `pr_state` table.
- **[P19–P25]** *Polish features.* Search highlighting, `[[` autocomplete, image/PDF/external-link rendering, the graph view with react-force-graph-2d, file uploads with multipart parsing, the rectangle-node redesign with freeze-on-drag, and `localStorage`-backed layout persistence.

We also used the model in a non-generative mode — asking it to read and explain the existing code, recommend an implementation strategy (e.g. "what's the simplest way to add a tsvector index?"), and audit the proposal PDF against our code to identify what was still missing.

6 Testing

Testing was primarily *interactive and end-to-end* rather than unit-test-heavy:

- **Database invariants checked via `psql`**, e.g. confirming the `search_vector` column was populated for every row, that the `page_rank` view contained one row per note, and that "Operating Systems" (the most-linked-to note in the seed) had the highest normalized score (1.0).
- **API smoke tests with `curl`** for every new endpoint — `/api/search?q=memory` returns "Memory Management" first, with "Operating Systems" beating "Virtual Memory" *only* once we add the graph component (demonstrating the combined ranking is doing what the proposal claimed); `/api/orphans` returns `[]` for the well-connected seed; `/api/path` from "Concurrency" to "Databases" returns the expected 4-hop chain.
- **Upload validation:** POSTed a known-good PNG and got back the served URL with a `200 OK`; POSTed a `.txt` and confirmed the 400 with the allowed-types list.
- **Manual UI walkthrough** of: create/edit/delete a note, alias links rendering correctly, search highlighting on click-through from results, autocomplete keyboard navigation, image and PDF upload & embed, graph drag/zoom/click, layout persistence across page reloads, "Reset layout" returning to auto-layout.
- **Build sanity:** every change passes `npm run build` (CRA's compile + ESLint pass) and `python -c "import main"` on the backend.

7 What's Not Done (Future Work)

The proposal also mentioned scaling the dataset to "a few hundred notes" and producing quantitative benchmarks (`EXPLAIN ANALYZE` with vs. without the GIN index, with vs. without precomputed PageRank). The current seed is 10 notes; benchmarking against a larger corpus is the natural next step and is the main thing left from the original plan.

8 Submission Notes

The repository is public at `github.com/Mehul-Borad/Link-Notes-DBIS` (link at the top of this report). Per-file diffs against the initial commit are included as `*.diff` files in the submitted archive (generated with `git diff <initial-sha> - <path>`). The full prompt log is `PROMPT_LOG.md` at the repo root; section 5 above ties prompt numbers to the features they produced.